

Testing P0 – commandLine

Marks distribution for testing exercise 3		TOTAL
Alba Llorente	10	10

1) Write at least the pseudocode corresponding to the identified method or method

Code is located in commandLine branch of the main project

2) Identify the test units.

Each method would be considered a test unit. Therefore, we would have:

- readInt(), readPositiveInt(), readDouble(), readPositiveDouble(), readDate(), readString(), readLine(), printLine(), print()

3) Identify the variables that must be considered to test the relevant methods included in the test units.

For readInt, readPositiveInt(), readDouble, readPositiveDouble(), readDate and readString the variable to consider is the string **input**.

For printLine and print the variable to consider is **output**

4) Identify the test values for each of the above variables, which will constitute the corresponding test cases using the three techniques discussed in theory (equivalence classes, boundary value analysis, and error guessing), specifying for each value obtained which technique has been used.

These are the test values for each variable of each method:

readInt():

Variable	Partitions	Limit	Guess
input	"2"	MAXINT	" 3 " (with spaces)
	"-6"	MININT	"45b"
	"2.5"		MAXINT + 1
	"hello"		MININT - 1
	" "		

readPositiveInt():

Variable	Partitions	Limit	Guess
input	"2"	MAXINT	" 3 " (with spaces)
	"-6"	MININT	"45b"
	"2.5"		MAXINT + 1

	“hello”		MININT - 1
	“ ” —		

readDouble():

Variable	Partitions	Limit	Guess
input	“2”	MAXDOUBLE	“ 3.66 “ (with spaces)
	“2.5”	MINDOUBLE	“3,66”
	“hello”		MAXDOUBLE + 1
	“ ” —		MINDOUBLE - 1

readPositiveDouble():

Variable	Partitions	Limit	Guess
input	“2”	MAXDOUBLE	“ 3.66 “ (with spaces)
	“2.5”	MINDOUBLE	“3,66”
	“hello”		MAXDOUBLE + 1
	“ ” —		MINDOUBLE - 1

readDate():

Variable	Partitions	Limit	Guess
input	“08/04/2005”	01/04/2025	“01/04/2025 “ (with spaces)
	“ ” —	31/03/2025	“03-24-1950”
	“hello”	00/04/2025	
		32/04/2025	
		06/01/2025	
		06/13/2025	
		06/00/2025	
		06/04/-0001	

readBoolean():

Variable	Partitions	Limit	Guess
input	“true”		“1”
	“false”		“yes”
	“TRUE”		
	“tRuE”		

readString():

Variable	Partitions	Limit	Guess
input			"abcd"
			"123"
			"_"

Here there's just some guess values as the readLine() reads everything u insert.

println():

Variable	Partitions	Limit	Guess
output	"abcd"		
	"123"		
	"_"		

Here there's just some guess values as println() prints every string u insert.

print():

Variable	Partitions	Limit	Guess
output	"abcd"		
	"123"		
	"_"		

Here there's just some guess values as print() prints every string u insert.

5) Calculate the maximum possible number of test cases that could be generated from the test values (combinatorial).

For calculating the maximum number of test cases we have to combine all the possible inputs for every variable of each method, but as we only have 1 variable for each method the result is just sum all. So we have 11 10 ,13 ,3 ,3, 2

6) Define a set of test cases to fulfill each use (each value at least once).

As we only have 1 variable per method its just one test case for each value of the method. Therefore we would have this number of test cases:

- readInt():11
- ReadPositiveInt(): 11

- readDouble(): 10
- ReadPositiveDouble(): 11
- readDate(): 13
- readString(): 3
- printLine(): 3
- print(): 2

7) Define test sets to achieve pairwise coverage using the algorithm explained in class. The results can be verified using the PICT program5 .

Again as we only have 1 variable per method its just the same as the each value

8) For code sections involving decisions, propose a set of test cases to achieve decision coverage.

Decision coverage is met when a decision is evaluated to true and false at least one. As each method looks like this:

```
try{
    parse()
    If( num >= 0 )flag=false
} catch (Exception) {
    Flag=true
}
```

For a correct decision coverage we just need to have one execution that goes into the try but not into the catch and one that does enter into the catch.

Input	Condition	Decision
A	A is valid input?	parse is correct?
"3"	TRUE	TRUE
"abcd"	FALSE	FALSE

For readPositiveInt() and readPositiveDouble() it changes a bit as they look like this:

```
try{
    parse()
    flag=false
```

```

} catch (Exception) {

    Flag=true

}

```

So each decision is evaluated to true and false at least one

Input	C1	C2	D1	D2
A	A is valid input?	A is positive?	parse is correct?	A >= 0
"3"	TRUE	TRUE	TRUE	TRUE
"-3"	TRUE	FALSE	TRUE	FALSE
"abcd"	FALSE	-	FALSE	-

9) For code sections involving decisions, propose a set of test cases to achieve MC/DC coverage.

This is met when every possible value of a condition determines the output of the decision at least once. As we only have one variable the table would be the same as in decision coverage for every method.

To clarify, for the methods reading positive numbers:

Input	C1	C2	D1	D2	
A	A is valid input?	A is positive?	parse is correct?	A >= 0	
"3"	TRUE	TRUE	TRUE	TRUE	Affects both turning true
"-3"	TRUE	FALSE	TRUE	FALSE	Affects D2 turning false
"abcd"	FALSE	-	FALSE	-	Affects D1 turning false

10) Comment on the results regarding the number of test cases obtained in sections 4, 5, and 6: what can be said about the coverage achieved? To what extent does the implementation of the program influence the design and implementation of the test cases??

As the methods only depend on one input variable, the different techniques don't increase nor reduce the number of test cases, resulting in the same number of cases as

the values decided on section 4. In addition the decision is very simple for this program resulting in the same table for decision coverage and MC/DC coverage.

We can also see that the implementation of the program influences a lot the design and implementation of test cases. For example the use of `nextLine()` and `parse` requires us to include invalid inputs and capture the exceptions. The use of `while` makes the code to keep asking inputs until the `parse` goes well, that also affects the implementation of test cases as we have to always have one valid input at the end for the loop to end.

Final test cases:

Knowing this, the test cases to be implemented are the following:

`readInt():`

Input value	Expected result
"2\n"	<code>AssertEquals(input,2) == TRUE</code>
"-6\n"	<code>AssertEquals(input,-6) == TRUE</code>
"2.5\n3\n"	<code>AssertEquals(input,3) == TRUE</code>
"hello\n10\n"	<code>AssertEquals(input,10) == TRUE</code>
" \n8\n"	<code>AssertEquals(input,8) == TRUE</code>
"45b\n4\n"	<code>AssertEquals(input,4) == TRUE</code>
"2147483647\n"	<code>AssertEquals(input, 214748364) == TRUE</code>
"-2147483648\n"	<code>AssertEquals(input, -2147483648) == TRUE</code>
"2147483648\n0\n"	<code>AssertEquals(input, 0) == TRUE</code>

`readDouble():`

Input value	Expected result
"2\n"	<code>AssertEquals(input,2) == TRUE</code>
"2.5\n"	<code>AssertEquals(input,-6) == TRUE</code>
"hello\n3.14\n"	<code>AssertEquals(input,3.14) == TRUE</code>
" \n10.5\n"	<code>AssertEquals(input,10.5) == TRUE</code>
"1.7E308\n"	<code>AssertEquals(input, 1.7E308) == TRUE</code>
"1.8E308\n "	<code>AssertEquals(input, -2147483648) == TRUE</code>
"-1.8E308\n"	<code>AssertEquals(input, 2147483648) == TRUE</code>

`readDate():`

Input value	Expected result
-------------	-----------------

"08/04/2005\n"	AssertEquals(input, 08/04/2005) == TRUE
"hello\n05/03/2020\n"	AssertEquals(input, 01/01/2020) == TRUE
" \n05/03/2020\n"	AssertEquals(input, 01/01/2020) == TRUE
"01/04/2025\n"	AssertEquals(input, 01/04/2025) == TRUE
"31/03/2025\n"	AssertEquals(input, 31/03/2025) == TRUE
"00/04/2025\n 05/03/2020\n"	AssertEquals(input, 05/03/2020) == TRUE
"32/04/2025\n 05/03/2020\n"	AssertEquals(input, 05/03/2020) == TRUE
"06/01/2025\n 05/03/2020\n"	AssertEquals(input, 05/03/2020) == TRUE
"06/13/2025\n 05/03/2020\n"	AssertEquals(input, 05/03/2020) == TRUE
"06/00/2025\n 05/03/2020\n"	AssertEquals(input, 05/03/2020) == TRUE
"06/04/-0001\n 05/03/2020\n"	AssertEquals(input, 05/03/2020) == TRUE
"01/04/2025 \n05/03/2020\n "	AssertEquals(input, 05/03/2020) == TRUE
"03-24-1950\n03/24/1950\n"	AssertEquals(input, 03/24/1950) == TRUE

readBoolean():

Input value	Expected result
"true\n"	AssertEquals(input,) == TRUE
"false\n"	AssertEquals(input,-6) == TRUE
"TRUE\n"	AssertEquals(input,3.14) == TRUE
"tRuE\n"	AssertEquals(input,10.5) == TRUE
"yes\ntrue\n"	AssertEquals(input, 1.7E308) == TRUE
"1\ntrue\n"	AssertEquals(input, -2147483648) == TRUE

readString():

Input value	Expected result
"abc\n"	AssertEquals(input, "abc") == TRUE
"123\n"	AssertEquals(input, "123") == TRUE
" \n"	AssertEquals(input," \n") == TRUE

printLine():

Input value	Expected result
"abc"	AssertEquals(input, "abc\r\n") == TRUE

print():

Input value	Expected result
"abc"	AssertEquals(input, "abc") == TRUE